QUANTUM-RESILIENT AUTHENTICATED COMMUNICATION FOR CONNECTED AUTONOMOUS VEHICLES USING POST-QUANTUM CRYPTOGRAPHY

A PROJECT REPORT

Submitted by

HARINI R **[Register No.]**
ANANYALAKSHMI V K **[Register No.]**
ADHITHYA K **[Register No.]**
RATISH S A **[Register No.]**

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
in
INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY
COLLEGE OF ENGINEERING GUINDY
ANNA UNIVERSITY, CHENNAI 600 025

[MONTH] 2026

BONAFIDE CERTIFICATE

Certified that this project report titled **"Quantum-Resilient Authenticated Communication for Connected Autonomous Vehicles using Post-Quantum Cryptography"** is the bonafide work of **Harini R, Ananyalakshmi V K, Adhithya K and Ratish S A**, who carried out the project work under my supervision.

SIGNATURE

Dr. [Name]
Head of the Department
Department of Information Technology
College of Engineering Guindy
Anna University, Chennai 600 025

SIGNATURE

Dr. Vijayalakshmi M
Supervisor
Department of Information Technology
College of Engineering Guindy
Anna University, Chennai 600 025

ACKNOWLEDGEMENT

[To be written by the team.]

ABSTRACT

Connected autonomous vehicles exchange safety messages with roadside units (RSUs) and edge servers. Those messages are authenticated today with ECDSA and keyed with ECDH, and Shor's algorithm breaks both on a large quantum computer. Recorded traffic can be decrypted later, so the transition to post-quantum cryptography has to start before such machines exist. In 2024 NIST standardised ML-KEM (FIPS 203) for key establishment and ML-DSA (FIPS 204) for signatures. Both are far larger than the elliptic-curve schemes they replace, and vehicular radio is slow and shared.

This project implements, end to end, a post-quantum security framework for vehicle-to-everything (V2X) communication based on the architecture of Aslam, Bhardwaj and Chaudhary (2025). It covers a Trusted Authority with ML-DSA certificates and a signed revocation list; a four-flight, mutually authenticated ML-KEM/ML-DSA handshake with transcript-bound key derivation; AES-256-GCM records with replay protection; criticality-based routing through edge servers; and an Adaptive Grouping Score PBFT (AGS-PBFT) consensus layer, with view change, over a hash-linked ledger. While implementing the published design we found and corrected defects that prevent it from working as written, among them a consensus layer that signs with a KEM, which has no signing operation, and a handshake in which only the server authenticates. We also add certificate caching, which sends 32-byte certificate references instead of 5.3 KB ML-DSA certificates.

The system runs as 19 separate processes over TCP, in an ns-3 IEEE 802.11p simulation with mobility for 50–200 vehicles, and in an evaluation suite covering five parameter sets, a classical baseline, 21 attacks and the consensus protocol. With a native implementation a full post-quantum handshake costs about 4.77 ms of computation, but 20,779 bytes (17 radio frames) against 1,043 bytes classically. In ns-3 the classical handshake completes 100 % of the time even at 200 vehicles, while the post-quantum handshake falls to 74 % at 100 vehicles and 27 % at 200. The limit is channel capacity, not computation. Inside an established session a beacon costs the same 135-byte AES-GCM record for every suite. Certificate caching halves the handshake (10,173 bytes) and raises completion to 98 % at 100 vehicles. AGS-PBFT sends 73–86 % fewer messages than PBFT but is not faster, and the safety path never waits for it: alerts reach vehicles in about 5 ms. All 21 attacks were rejected.

CONTENTS

Bonafide Certificate i

Acknowledgement ii

Abstract iii

List of Abbreviations ix

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem statement . . . 2
- 1.3 Objectives . . . 2
- 1.4 Contributions . . . 3
- 1.5 Scope . . . 3
- 1.6 Organisation of the report . . . 3

2 Literature Survey 4
- 2.1 The quantum threat and the migration timeline . . . 4
- 2.2 Post-quantum standards . . . 4
- 2.3 V2X security today . . . 4
- 2.4 The base paper . . . 5
- 2.5 Handshake composition . . . 5
- 2.6 Byzantine fault-tolerant consensus . . . 6
- 2.7 Gap addressed by this project . . . 6

3 Background 7
- 3.1 Learning With Errors . . . 7
- 3.2 ML-KEM (FIPS 203) . . . 7
- 3.3 ML-DSA (FIPS 204) . . . 8
- 3.4 Symmetric building blocks . . . 8
- 3.5 PBFT . . . 9
- 3.6 IEEE 802.11p . . . 9

4 System Design 10
- 4.1 Architecture . . . 10
- 4.2 Threat model . . . 11
- 4.3 Registration and PKI . . . 11

4.4 Handshake . . . 11
4.5 Certificate caching . . . 13
4.6 Record layer . . . 14
4.7 Defects found and corrected . . . 14
4.7.1 In the base paper . . . 14
4.7.2 In our Review 1 design . . . 15
4.8 Edge routing . . . 15
4.9 AGS-PBFT . . . 15
4.9.1 Normal case . . . 15
4.9.2 Scores and regrouping . . . 16
4.9.3 Decisions where the paper is undefined . . . 16
4.9.4 View change . . . 16
4.10 Ledger . . . 16

5 Implementation **18**
5.1 Software structure . . . 18
5.2 Cryptographic suites and backends . . . 19
5.3 Distributed system . . . 19
5.4 ns-3 simulation . . . 20
5.4.1 Method . . . 20
5.4.2 Simulator pitfalls found and corrected . . . 20
5.4.3 Certificate caching in the simulation . . . 21
5.4.4 Replay viewer . . . 21
5.5 Evaluation harness . . . 21
5.6 Testing . . . 22
5.7 Bugs found by our own verification . . . 22

6 Results and Discussion **24**
6.1 Cryptographic cost . . . 24
6.1.1 Primitives . . . 24
6.1.2 Handshake size and compute . . . 25
6.1.3 Per-beacon budget . . . 25
6.2 Distributed system . . . 26
6.3 ns-3: 802.11p with mobility . . . 27
6.3.1 Handshake success and latency . . . 27
6.3.2 Other measures . . . 27
6.3.3 Certificate caching . . . 28
6.3.4 Variants and mitigations . . . 28
6.4 Consensus . . . 30

6.5 Security 32
6.6 Answers to the research questions 32

7 Conclusion and Future Work **35**
7.1 Conclusion 35
7.2 Limitations 35
7.3 Future work 36

References **37**

A Running the System **39**
A.1 Installation 39
A.2 Commands 39
A.3 Replay viewer 39

LIST OF FIGURES

4.1 Entities and links. The MEC sends alerts back through the RSUs before it submits a critical event to consensus. 10
4.2 The mutually authenticated handshake. T is the transcript of all four flights. 12

6.1 Handshake bytes by flight. The ServerHello carries the server's certificate, the ephemeral ML-KEM key and a signature. 25
6.2 Full handshake compute time, native and pure-Python backends (log scale). 26
6.3 Per-beacon cost against the 100 ms beacon interval. 27
6.4 Handshakes completed and aggregate channel airtime against density (mean of 3 seeds, bars show min–max). 29
6.5 Handshake completion time, median and 95th percentile (log scale). 29
6.6 Beacon slots with a live session, and the time from a critical event to the alert at the reporting vehicle. 30
6.7 Certificate caching: bytes per handshake for every suite, and its effect on DECK in ns-3. 31
6.8 Ablation: what EDCA priority and handshake backoff each contribute (DECK, 100 vehicles). 32
6.9 Messages, bytes and latency per request against network size. . . 33
6.10 Score dynamics with one wrong-result node and one silent node (16 nodes, regroup every 10 blocks). 34

A.1 The replay viewer: vehicles (dots) on the 1.5 km grid, RSUs (squares), handshakes in progress (dashed) and sessions (solid), with the channel airtime over time. The same run can be switched between the classical suite, DECK and DECK with certificate caching. 40

LIST OF TABLES

3.1 ML-KEM parameter sets (FIPS 203) 8
3.2 ML-DSA parameter sets (FIPS 204) and ECDSA P-256 for comparison 8

4.1 Defects in the base paper and their corrections 14
4.2 Corrections to the Review 1 handshake design 15
4.3 Design decisions for AGS-PBFT 17

5.1 Package structure 18
5.2 ns-3 scenario 20

6.1 Primitive operations (ms) 24
6.2 Key, signature, certificate and handshake sizes (bytes), with and without certificate caching 25
6.3 Compute time of one full handshake, both sides (ms) 26
6.4 Per-beacon cost, p99 crypto plus airtime (ms) 27
6.5 Handshakes completed (%) by vehicle density 28
6.6 Median handshake completion time (ms) 28
6.7 Other measures at 100 vehicles 30
6.8 Single-seed variants at 100 vehicles 31
6.9 AGS-PBFT against PBFT 32
6.10 Fault tolerance 33
6.11 Recovery after a leader crash 34
6.12 Attacks run against the implementation 34

A.1 Commands 39

LIST OF ABBREVIATIONS

AEAD	Authenticated Encryption with Associated Data
AES-GCM	Advanced Encryption Standard in Galois/Counter Mode
AGS-PBFT	Adaptive Grouping Score Practical Byzantine Fault Tolerance
BSM / CAM	Basic Safety Message / Cooperative Awareness Message
CAV	Connected Autonomous Vehicle
CRL	Certificate Revocation List
DHKEM	Diffie–Hellman based Key Encapsulation Mechanism (RFC 9180)
ECDH / ECDSA	Elliptic Curve Diffie–Hellman / Digital Signature Algorithm
EDCA	Enhanced Distributed Channel Access (802.11 QoS)
HKDF	HMAC-based Key Derivation Function
HNDL	Harvest Now, Decrypt Later
KEM	Key Encapsulation Mechanism
LWE / MLWE	Learning With Errors / Module Learning With Errors
MEC	Multi-access Edge Computing
ML-DSA	Module-Lattice-Based Digital Signature Algorithm (FIPS 204)
ML-KEM	Module-Lattice-Based Key-Encapsulation Mechanism (FIPS 203)
OBU	On-Board Unit
PBFT	Practical Byzantine Fault Tolerance
PKI	Public Key Infrastructure
PoP	Proof of Possession
PQC	Post-Quantum Cryptography
RSU	Roadside Unit
SHAKE	Secure Hash Algorithm KECCAK extendable-output function
TA	Trusted Authority
V2I / V2V / V2X	Vehicle-to-Infrastructure / -Vehicle / -Everything

CHAPTER 1

INTRODUCTION

1.1 Background

A connected autonomous vehicle (CAV) is in continuous radio contact with the infrastructure around it. It broadcasts its position, speed and heading about ten times a second, reports hazards such as a collision ahead or ice on the road, and receives warnings issued by roadside units (RSUs) and edge servers. Every one of these messages is safety-relevant: a forged "brake now" or a suppressed hazard warning can cause a crash. Vehicle-to-everything (V2X) security therefore rests on two guarantees, that a message really comes from the registered entity that claims to have sent it, and that it has not been altered or replayed.

Deployed V2X security, specified in IEEE 1609.2 [16], provides these guarantees with elliptic-curve cryptography: ECDSA signatures on messages and certificates, and ECDH where confidentiality is needed. Both rest on the elliptic-curve discrete-logarithm problem. Shor's algorithm [6] solves that problem, and integer factorisation, in polynomial time on a sufficiently large quantum computer. Such machines do not exist yet, but two facts make the problem urgent now. First, an adversary can record encrypted traffic today and decrypt it once the hardware arrives ("harvest now, decrypt later" [8]); a vehicle's movement history stays sensitive for years. Second, vehicles stay in service for fifteen years or more, so the cryptography fitted to a car sold today must still be secure in 2040.

In August 2024 NIST published the first post-quantum standards: ML-KEM (FIPS 203 [3]), a key-encapsulation mechanism derived from CRYSTALS-Kyber, and ML-DSA (FIPS 204 [4]), a signature scheme derived from CRYSTALS-Dilithium. Both are built on the Module Learning With Errors problem over lattices, for which no efficient quantum algorithm is known, and both run on ordinary processors. Their cost is size: an ML-DSA-65 public key is 1,952 bytes and a signature 3,309 bytes, against 65 and about 72 bytes for ECDSA P-256. On a 6 Mb/s IEEE 802.11p channel shared by every vehicle in range, that difference matters more than computation time.

1.2 Problem statement

Our base paper, Aslam, Bhardwaj and Chaudhary [1], proposes a quantum-resilient V2X framework: a Trusted Authority registers entities, vehicles establish keys with RSUs using Kyber, edge servers route events by criticality, and an Adaptive Grouping Score PBFT (AGS-PBFT) consensus layer records critical events on a ledger. The paper reports latency and message counts but leaves open the questions a deployment depends on:

1. Can the framework be implemented as specified with the standardised algorithms, and is the resulting protocol secure?
2. Do post-quantum handshakes and safety messages fit vehicular latency budgets (100 ms for periodic beacons) when computation *and* radio transmission are both measured?
3. How does a shared 802.11p channel behave as vehicle density grows, when every handshake carries post-quantum keys, signatures and certificates?
4. Does AGS-PBFT deliver the savings the paper claims over PBFT, and at what cost?

1.3 Objectives

1. Implement the complete framework (Trusted Authority, registration, mutually authenticated ML-KEM/ML-DSA handshake, record layer, edge routing, AGS-PBFT with view change, and a ledger) as a working distributed system.
2. Correct every defect in the published design that prevents it from working or makes it insecure, and document each correction.
3. Measure computation, message size and radio airtime for all NIST security levels and for a classical ECDH/ECDSA baseline run through the same protocol code.
4. Evaluate the protocol in ns-3 with IEEE 802.11p, fading and vehicle mobility from 50 to 200 vehicles.
5. Compare AGS-PBFT with PBFT using real signatures, and test both under Byzantine faults and leader crashes.
6. Test the implementation against concrete attacks.
7. Reduce the handshake's size, the dominant cost, with certificate caching.

1.4 Contributions

- **A complete, running implementation** (the `qrcav` package, about 7,600 lines of Python, including tests, and 870 lines of C++ for ns-3) that runs as 19 processes over TCP, inside ns-3, and as an evaluation suite, with 57 automated tests.
- **Corrections to the base paper** (D1–D5): a consensus layer that signs with a KEM, a circular key-derivation listing, a session key that hashes a value with itself, key escrow at the Trusted Authority, and a ledger platform that cannot host PBFT. **Corrections to our own Review 1 design** (F1–F7), the most serious being the absence of client authentication.
- **Design decisions where the paper is undefined** (D6–D9) that make AGS-PBFT implementable: regrouping when all scores are equal, how candidates earn promotion, how group size follows load, and how the leader's own vote counts.
- **Certificate caching** for the post-quantum handshake, with its security argument, which halves the handshake to 10,173 bytes.
- **A measured answer** to the questions above: per-message post-quantum protection fits the budget; channel capacity during handshakes does not scale; the cause is size, which certificate caching and exponential backoff after failed handshakes reduce.

1.5 Scope

The project uses ordinary (classical) hardware; post-quantum cryptography is software and needs no quantum devices. Out of scope are deployment on real on-board units, side-channel hardening beyond what the OpenSSL implementation provides, formal verification of the protocol, and pseudonymous certificates for location privacy.

1.6 Organisation of the report

Chapter 2 surveys related work. Chapter 3 explains the cryptographic and consensus building blocks. Chapter 4 presents the system design, the defects found and the corrections. Chapter 5 describes the implementation and the simulation methodology. Chapter 6 reports and discusses the results, and Chapter 7 concludes with limitations and future work.

CHAPTER 2

LITERATURE SURVEY

2.1 The quantum threat and the migration timeline

Shor [6] showed that a quantum computer factors integers and computes discrete logarithms in polynomial time, which breaks RSA, finite-field Diffie–Hellman and every elliptic-curve scheme. Grover's search algorithm [7] gives only a quadratic speed-up against symmetric primitives, so AES-256 and SHA-3 keep an ample margin. Mosca [8] framed the planning problem: if data must stay secret for x years and migration takes y years, systems are already exposed when $x + y$ exceeds the time until a cryptographically relevant quantum computer exists. Vehicles are a clear case, with long service lives and slow fleet-wide updates. NIST's transition guidance [5] proposes deprecating quantum-vulnerable public-key algorithms by 2030 and disallowing them by 2035.

2.2 Post-quantum standards

NIST's standardisation process selected CRYSTALS-Kyber [9] for key establishment and CRYSTALS-Dilithium [10] for signatures, published in 2024 as ML-KEM [3] and ML-DSA [4]. Both rest on Module-LWE [11, 12]. ML-KEM is IND-CCA2 secure through a variant of the Fujisaki–Okamoto transform [13], with implicit rejection: a malformed ciphertext yields a pseudorandom key instead of an error. ML-DSA follows the Fiat–Shamir-with-aborts paradigm, so signing repeats until a candidate passes a bound check; signing time is therefore variable, while verification is deterministic. Each standard defines three parameter sets (ML-KEM-512/768/1024, ML-DSA-44/65/87) mapped to NIST security categories.

2.3 V2X security today

IEEE 1609.2 [16] defines the security services of the WAVE stack used over IEEE 802.11p [17]: periodic safety messages are broadcast with an ECDSA P-256 signature and a certificate or certificate digest, and certificates are issued by a PKI. The design assumes small signatures. A post-quantum signature per

beacon multiplies the frame size by more than ten, which is why this project separates the one-off handshake from the per-beacon cost and evaluates both a session-based (AEAD) beacon design and a signed-broadcast design.

For the radio itself we rely on the vehicular channel model of Torrent-Moreno et al. [18], whose Nakagami fading parameters are the usual choice for 802.11p highway and urban studies, and on the ns-3 network simulator [19].

2.4 The base paper

Aslam, Bhardwaj and Chaudhary [1] propose a four-layer framework: vehicles at the user layer, RSUs and multi-access edge computing (MEC) servers at the edge, a consensus layer of validator nodes, and a cloud archive. A Trusted Authority registers every entity; vehicles and RSUs establish session keys with Kyber; edge servers send critical events to consensus and routine data to the cloud. Its main contribution is AGS-PBFT: nodes carry a score (starting at 100, $+1$ for agreeing with the committed result, -5 for disagreeing), only a *consensus set* of size $3f + 1$ votes, and every 50 requests nodes below $\mu - \sigma$ are demoted to a *candidate set* while candidates above $\mu + \sigma$ are promoted. The paper reports lower message counts than PBFT and an end-to-end latency evaluation in a vehicular simulator.

On implementation we found that the paper cannot be executed as written (Section 4.7): its consensus messages are to be "signed with Kyber", which has no signing operation; its key-exchange listing is circular; the Trusted Authority generates entities' private keys although the paper claims no key escrow; and Hyperledger Fabric 2.2, the platform it lists, orders transactions with Raft [15] and cannot run a PBFT-family protocol.

2.5 Handshake composition

Olushola and Meenakshi [2] combine ML-KEM-1024 with ML-DSA-65 in an authenticated key exchange for vehicular networks; our Review 1 slides adopted the same pairing, which we call the *DECK* suite. The pattern of an ephemeral KEM key signed by a long-term signature key, with keys derived from a hash of the whole transcript, follows TLS 1.3 [20]. Our key schedule uses HKDF [23]; the record layer uses AES-GCM [24] with an anti-replay window in the style of IPsec [25]; the classical baseline uses the DHKEM construction of HPKE [22] so that ECDH fits the same KEM interface. Certificate caching follows the idea of TLS cached information [21].

2.6 Byzantine fault-tolerant consensus

PBFT [14] tolerates f Byzantine replicas out of $3f + 1$ with three phases (pre-prepare, prepare, commit) and a view-change protocol that replaces a faulty leader. Its message complexity is quadratic in the number of replicas, which motivates variants such as AGS-PBFT that restrict voting to a subset chosen by reputation.

2.7 Gap addressed by this project

The base paper evaluates its framework by simulation and reports latency and message counts, but it cannot be executed as written, and it folds the one-off handshake into the per-message latency. What is missing is a complete system in which the standardised algorithms run through a real protocol, with sizes and processing times measured from that code and then carried over a simulated 802.11p channel with mobility, alongside a classical baseline that uses the same protocol code. This project provides that, together with a working AGS-PBFT and a measured comparison against PBFT.

CHAPTER 3

BACKGROUND

3.1 Learning With Errors

Given a public matrix A and a vector $b = As + e \pmod{q}$, where s is secret and e is a vector of small random "errors", the Learning With Errors (LWE) problem [11] asks for s. Without e this is linear algebra. With it, the best known classical and quantum algorithms take exponential time, and Regev showed that solving random instances is as hard as worst-case lattice problems. Module-LWE [12] replaces the entries of A with polynomials in $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$, which shrinks keys and allows fast multiplication with the number-theoretic transform. Both ML-KEM and ML-DSA use $n = 256$; ML-KEM uses $q = 3329$ and ML-DSA uses $q = 8\,380\,417$.

3.2 ML-KEM (FIPS 203)

A key-encapsulation mechanism has three algorithms:

$$\begin{aligned} (ek, dk) &\leftarrow \mathsf{KeyGen}() \\ (K, c) &\leftarrow \mathsf{Encaps}(ek) \\ K &\leftarrow \mathsf{Decaps}(dk, c) \end{aligned}$$

Encaps chooses a random 32-byte message m, derives $(K, r) = G(m \parallel H(ek))$, and encrypts m under ek with randomness r. Decaps decrypts to m', re-derives (K', r'), re-encrypts and compares. If the ciphertexts differ it returns a pseudo-random key derived from a secret value z instead of an error (implicit rejection), so an attacker learns nothing from malformed ciphertexts.

Two properties matter for protocol design. First, **a KEM is not Diffie–Hellman**: only the receiver has a key pair, and the sender's contribution is fresh randomness. A KEM therefore authenticates neither party by itself; authentication must come from signatures. Second, **a KEM has no signing operation**, which is the root of defect D1 in the base paper.

The encapsulation key is k polynomials of 256 coefficients at 12 bits each, plus a 32-byte seed for A: for ML-KEM-1024, $4 \times 384 + 32 = 1{,}568$ bytes. The shared secret is always 32 bytes.

Table 3.1: ML-KEM parameter sets (FIPS 203)

Parameter set	k	category	ek (B)	dk (B)	ciphertext (B)
ML-KEM-512	2	1	800	1,632	768
ML-KEM-768	3	3	1,184	2,400	1,088
ML-KEM-1024	4	5	1,568	3,168	1,568

3.3 ML-DSA (FIPS 204)

ML-DSA signs with a Fiat–Shamir-with-aborts construction. The signer picks a masking vector y, computes $w = Ay$, derives a challenge c from the hash of the message representative μ and the high bits of w, and outputs $z = y + cs_1$ only if z leaks nothing about the secret s_1; otherwise it starts again. The verifier recomputes the high bits of $Az - ct_1$ from the public key, hashes them with μ, and checks the result against c. The message is never inside the signature: the verifier needs the message, the signature and the public key. FIPS 204 adds a *context string* to the signed message; we use `qrcav-v1` for domain separation.

Table 3.2: ML-DSA parameter sets (FIPS 204) and ECDSA P-256 for comparison

Scheme	category	public key (B)	signature (B)
ML-DSA-44	2	1,312	2,420
ML-DSA-65	3	1,952	3,309
ML-DSA-87	5	2,592	4,627
ECDSA P-256	–	65	70–72

3.4 Symmetric building blocks

Hash. H is SHAKE-256 over length-prefixed inputs, so $H(ab, c) \neq H(a, bc)$.

Key derivation. HKDF [23] with HMAC-SHA3-256 extracts a pseudorandom key from the KEM shared secret and expands it into independent keys and IVs for each direction, each with its own label.

Records. AES-256-GCM [24] provides confidentiality and integrity. Its one hard rule is that a (key, nonce) pair must never repeat: reuse reveals the authentication key and allows forgeries. We derive the nonce as $v \oplus (0^{32} \parallel \mathrm{be64}(ctr))$ from a per-direction IV v and a record counter.

3.5 PBFT

PBFT [14] runs on $n \geq 3f + 1$ replicas, one of which is the leader (primary) for the current view. The leader assigns a sequence number to a request in a PRE-PREPARE; replicas broadcast PREPARE, and after $2f + 1$ matching prepares broadcast COMMIT; after $2f + 1$ commits they execute and reply. Any two quorums of $2f + 1$ intersect in at least one honest replica, which gives *safety* (no two honest replicas commit different values at one sequence number). *Liveness* needs the leader to make progress; if it does not, replicas time out and run a view change that installs the next leader and carries prepared requests across. Messages per request grow as $O(n^2)$.

3.6 IEEE 802.11p

IEEE 802.11p [17] is Wi-Fi adapted for vehicles: 10 MHz channels at 5.9 GHz, typically 6 Mb/s for safety messages, no association (communication outside the context of a BSS), and EDCA access categories that let safety traffic (AC_VO) win the channel before bulk traffic (AC_BE). Periodic beacons are link-layer broadcasts with no acknowledgement or retransmission. A 1,400-byte payload occupies the channel for about 2 ms at 6 Mb/s, so a 20 KB handshake costs about 30 ms of airtime before any contention or loss.

CHAPTER 4

SYSTEM DESIGN

4.1 Architecture

Figure 4.1 shows the entities and the links between them. Every link except registration is a mutually authenticated ML-KEM/ML-DSA session carrying AES-256-GCM records.

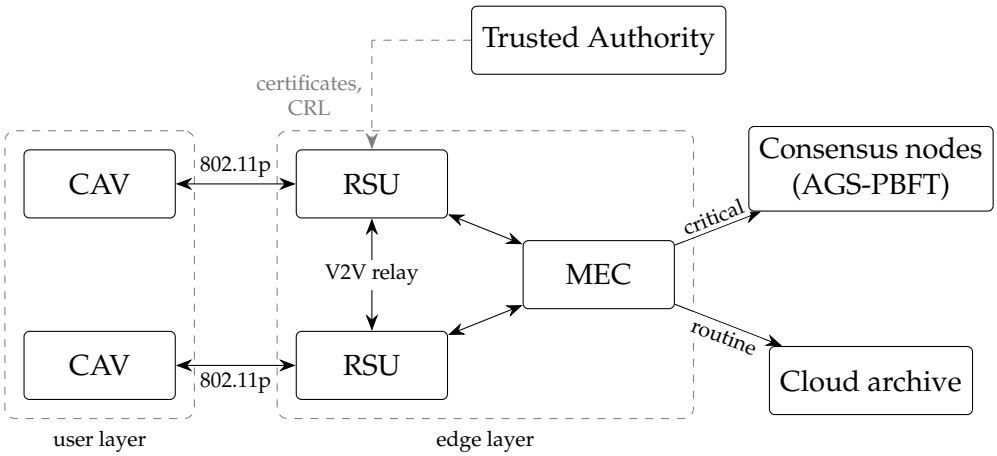


Figure 4.1: Entities and links. The MEC sends alerts back through the RSUs before it submits a critical event to consensus.

Trusted Authority (TA). Generates the root key pair, certifies the public keys of all entities, keeps the registry database and signs the certificate revocation list (CRL).

CAV. A vehicle. It holds its own ML-DSA key pair and certificate, opens a session with the nearest RSU, sends beacons at 10 Hz and reports events.

RSU. Terminates vehicle sessions, relays V2V messages to neighbouring RSUs, and forwards events to the MEC over the wired backhaul.

MEC. The edge server. It routes events by criticality: a critical event is first turned into an alert sent to every RSU (and so to every vehicle), *then* submitted to consensus; routine data goes to the cloud.

Consensus nodes. Eight validators running AGS-PBFT. Each committed block carries the event, the agreed validation result, a commit certificate, the score table and the consensus membership.

Cloud. An archive for routine data.

4.2 Threat model

The adversary controls the radio: it can record, drop, delay, replay, modify and inject any frame, and it can run its own transmitters. It may obtain copies of any certificate (they are public), run a rogue certificate authority, and hold credentials that the TA has since revoked. Up to f consensus nodes may be Byzantine (arbitrary behaviour, including colluding on a wrong result), and a leader may crash. In the long term the adversary has a quantum computer and replays recorded traffic to it. The TA's private key and the private keys of honest entities are not compromised.

4.3 Registration and PKI

1. The entity generates its own ML-DSA key pair. The TA never sees a private key (this removes the key escrow of defect D4).
2. The entity sends $\{id, role, pk, alg\}$ and a proof of possession: a signature over the request with the new private key. Without it an attacker could have someone else's public key certified under its own name.
3. The TA checks the identity against its roster (the identities vetted out of band, such as manufacturer records and the RSU deployment list), the algorithm, the key length and the proof, and returns

$$Cert = \mathsf{Sign}(sk_{TA},\ \{id, role, pk, alg, serial, nb, na\}).$$

4. The entity verifies the certificate with pk_{TA} before using it.

The certificate binds the *role*, so a vehicle cannot present itself as an RSU, and a validity period and serial number, so it can expire and be revoked. An ML-DSA-65 certificate is 5,355 bytes. Revocation is a TA-signed CRL holding a version number, an issue time and the revoked serial numbers; nodes accept only a newer, validly signed CRL and close any open session with a revoked peer.

4.4 Handshake

Figure 4.2 shows the four-flight handshake between a client (initiator, usually a vehicle) and a server (responder, usually an RSU).

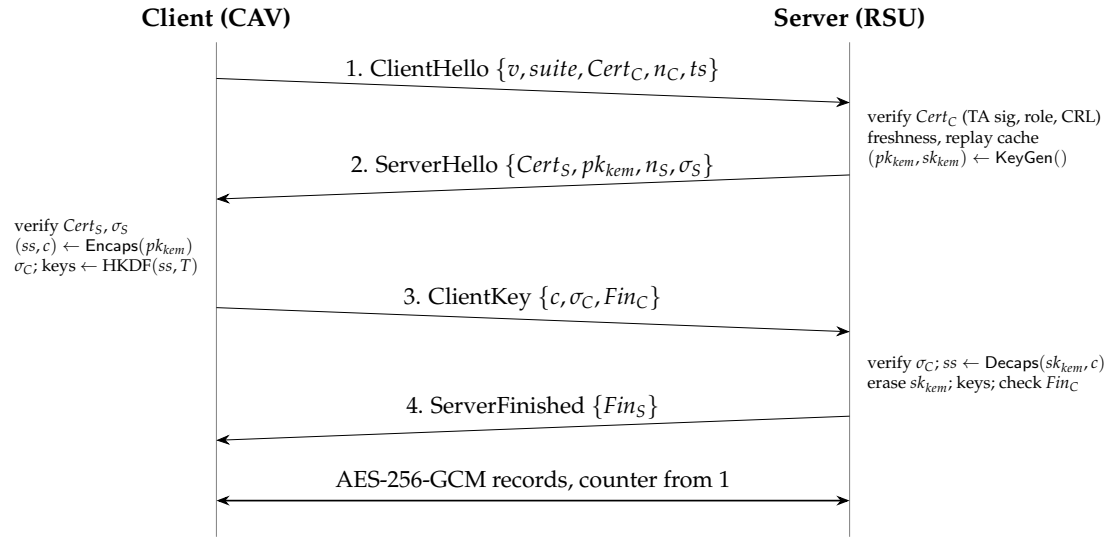


Figure 4.2: The mutually authenticated handshake. T is the transcript of all four flights.

$$
\begin{aligned}
\sigma_S &= \mathsf{Sign}(sk_S,\ \text{“sh”} \parallel H(CH, SH_{body})) \\
\sigma_C &= \mathsf{Sign}(sk_C,\ \text{“ck”} \parallel H(CH \parallel SH \parallel CK_{body})) \\
T_{hash} &= H(CH \parallel SH \parallel CK_{body} \parallel \sigma_C) \\
(k_{c2s}, v_{c2s}, k_{s2c}, v_{s2c}) &= \text{HKDF}(ss, T_{hash}) \\
Fin_C &= \text{AES-GCM}(k_{c2s}, v_{c2s} \oplus 0, H(\text{“client_finished”}, T_{hash}))
\end{aligned}
$$

Design properties:

- **Forward secrecy.** The server's KEM key pair is ephemeral and erased after decapsulation, so a later compromise of long-term signing keys does not reveal past session keys.
- **Mutual authentication.** σ_S binds the server's identity to the client's fresh nonce and to pk_{kem}; σ_C binds the client's identity to the whole transcript including the ciphertext c.
- **Key confirmation.** Because ML-KEM rejects implicitly, a tampered c silently produces a different key. The Finished messages detect it: each side proves it derived the same keys.
- **Downgrade protection.** The suite is in the ClientHello, must match exactly, and is covered by both signatures.
- **Uniform abort.** Every failed check raises the same error with no detail, so a probing attacker cannot tell which check failed.
- **Sans-IO.** The handshake state machines consume and return bytes and

never touch a socket, so the same code runs over TCP, in memory for tests and benchmarks, and supplies exact message sizes to ns-3.

4.5 Certificate caching

With ML-DSA-65 each certificate is 5,355 bytes, and the full handshake carries two of them: about half of its 20,779 bytes. Peers usually hold these certificates already. RSU certificates are few and long-lived, so they can be distributed to vehicles in advance with the CRL. A vehicle's certificate can be kept, after its first handshake, in an edge directory at the MEC that every RSU reaches over the wired backhaul.

When a peer is known to hold a certificate, the handshake sends a 32-byte reference $cref = H(\text{“qrcav-cert”}, Cert)$ instead, in the spirit of TLS cached information [21]:

1. ClientHello carries $cref_C$ instead of $Cert_C$ once the vehicle's certificate is in the directory, and $known = H(Cert_S)$ when the vehicle holds the RSU's certificate.
2. If *known* matches, ServerHello carries $cref_S$ instead of $Cert_S$.
3. If the RSU cannot resolve $cref_C$ (locally or from the directory) it answers *HelloRetry* $\{n_C\}$, and the client resends its ClientHello with the full certificate and a new nonce.

The security argument:

- **Checks are not cached.** A cached certificate is verified in full on every handshake (TA signature, validity, role, expected identity, current CRL), so revocation still takes effect at the next CRL refresh.
- **The reference is bound by both signatures.** *cref* is a collision-resistant hash of the exact certificate bytes and lies inside the messages that σ_C and σ_S cover. Claiming another vehicle's cached certificate fails σ_C; replacing the server's reference fails σ_S.
- **The directory is not trusted.** A fetched certificate must hash to the requested reference and is verified against pk_{TA}.
- **The cache cannot be filled by strangers.** A server caches a client certificate only after a handshake with its owner has completed, and the cache is a bounded LRU.
- **HelloRetry is unauthenticated, and that is acceptable.** It must echo the client's nonce and is honoured once per handshake. Forging it only forces

the full-certificate path, the same property as TLS 1.3's HelloRetryRequest [20].

For DECK the handshake shrinks from 20,779 to 10,173 bytes (51 % smaller) and from 17 to 10 radio frames. The cached ClientHello is 162 bytes, one frame.

4.6 Record layer

$$record = ctr\ (8\,\mathrm{B}) \parallel \mathrm{AES\text{-}256\text{-}GCM}(k_{dir},\ nonce,\ pt,\ aad = dir \parallel ctr)$$
$$nonce = v_{dir} \oplus (0^{32} \parallel \mathrm{be64}(ctr))$$

Counter 0 is reserved for the Finished message and application data starts at 1, so no (key, nonce) pair is ever reused. A 64-record sliding window accepts reordered records once and rejects replays; it is checked *before* decryption, so a replay flood costs a lookup rather than an AES operation. A session refuses to send more than 2^{32} records.

4.7 Defects found and corrected

4.7.1 In the base paper

Table 4.1: Defects in the base paper and their corrections

	Defect	Correction
D1	Consensus messages are signed and verified "using Kyber", a KEM with no signing operation.	ML-DSA signatures on every consensus message.
D2	The encapsulation listing defines k_i in terms of itself, and decapsulation uses k_i, which the receiver does not have.	The standard `Encaps`/`Decaps` interface.
D3	The session key is $H(k_i \parallel k'_j)$, a value hashed with itself, since a correct KEM gives $k_i = k'_j$.	HKDF over the shared secret and the hash of the whole transcript.
D4	The TA generates entities' private keys (key escrow) although the paper claims "no key escrow".	Entities generate their own keys; the TA certifies public keys only, with proof of possession.
D5	Hyperledger Fabric 2.2 orders with Raft and cannot host a PBFT-family protocol.	A ledger implemented directly, driven by AGS-PBFT.

4.7.2 In our Review 1 design

Table 4.2: Corrections to the Review 1 handshake design

	Review 1	Now, and why
F1	The server signs $T \parallel pk_{kem}$, but nothing initialises T.	A ClientHello with a fresh nonce and timestamp opens T, so a recorded ServerHello cannot be replayed.
F2	Only the server signs.	The client signs the transcript. Otherwise anyone holding a copy of a vehicle's (public) certificate can open a session as that vehicle.
F3	Finished uses counter 0; where data starts is unspecified.	Data starts at counter 1: no GCM nonce reuse.
F4	Key generation shown as $sk_{kem} \leftarrow (s, e, pk_{kem})$.	KeyGen(); FIPS 203's dk is $(s, ek, H(ek), z)$.
F5	The certificate signs only $ID \parallel pk$.	Role, serial and validity are bound too.
F6	No downgrade protection.	Suite in the ClientHello, matched exactly, signed.
F7	Abort reasons unspecified.	One uniform error.

4.8 Edge routing

Each event carries a criticality level set by the reporting vehicle, and the MEC routes on it. A critical event (collision ahead, pedestrian on the road, ice) is broadcast as an alert to every RSU, which re-encrypts it for each vehicle's session, and only then submitted to consensus for the ledger. A routine event (a pothole report, road condition) goes to the cloud archive. Safety warnings therefore never wait for a consensus round.

4.9 AGS-PBFT

4.9.1 Normal case

1. The MEC sends a signed REQUEST to the leader, $R[v \bmod |R|]$, where R is the sorted consensus set.
2. The leader sends a signed PRE-PREPARE to the consensus set and to the candidates.

3. Consensus nodes broadcast PREPARE. On $2f + 1$ matching prepares each node *validates* the event (fields present, reporter's TA certificate valid, event not already recorded) and sends its result in a RESPONSE to the leader. Candidates send a SHADOW-RESPONSE that is scored but not counted towards the quorum.
4. On $2f + 1$ matching responses the leader builds a block containing the request, the agreed result, the signed responses as a commit certificate, the updated scores and the membership, signs it, sends COMMIT to every node and REPLY to the MEC.
5. Every node re-verifies the block (hash link, leader signature, commit certificate, recomputed scores and membership) before appending it.

4.9.2 Scores and regrouping

A node gains $+1$ if its result matches the committed one and loses 5 if it differs. Every *regroup interval* (50 blocks in the paper), nodes compute μ and σ over all scores; consensus nodes below $\mu - \sigma$ are demoted and candidates above $\mu + \sigma$ promoted, and the set is resized to $3f + 1$. Because scores and membership are written into each block and re-derived by every node, all nodes agree on the consensus set without extra messages.

4.9.3 Decisions where the paper is undefined

4.9.4 View change

A consensus node that has seen a request but not its commit within a timeout broadcasts VIEW-CHANGE$\{v + 1, h, head, P\}$, where P holds its prepared certificates above its ledger height h. On $2f + 1$ view changes the new leader computes the set O of requests to re-issue (the highest-view prepared request for each sequence number, or a no-op for a gap) and sends NEW-VIEW$\{v + 1, V, O\}$. Replicas verify the signatures, recompute O, and on a match enter the new view. The timeout doubles after each failed view. Messages for a future view are buffered, because a pre-prepare can overtake the NEW-VIEW that precedes it.

4.10 Ledger

A block is $(B_ID, Pr_B_Hash, T_Block)$ as in the paper, plus view, sequence number, signed request, agreed result, commit certificate, score table, membership, leader identity and leader signature. Any party holding pk_{TA} can replay a

Table 4.3: Design decisions for AGS-PBFT

	Gap in the paper	Decision
D6	All nodes start at 100, so at the first regroup $\sigma = 0$ and $\mu - \sigma = \mu + \sigma$: every node is both above and below threshold.	Skip regrouping while $\sigma = 0$.
D7	Candidates never take part, so they can never gain score or be promoted.	Candidates validate as shadows and are scored. Absent nodes score 0, not -5, so a Byzantine leader cannot punish honest nodes by omitting their responses.
D8	"Group size is adjusted based on network activity" is not specified.	Request rate r over the interval, from request timestamps so every node computes the same value: f falls linearly from f_{max} at $r \leq 2/\text{s}$ to 1 at $r \geq 20/\text{s}$.
D9	$2f + 1$ prepares are required, but the leader sends none, so the quorum is unreachable with f faulty backups.	The leader also sends a PREPARE for its own proposal.

ledger from genesis and check every hash link, signature, certificate, score and membership change.

CHAPTER 5

IMPLEMENTATION

5.1 Software structure

The system is a Python package, `qrcav`, with a strict layering: lower layers never import upper ones, and the protocol code performs no I/O.

Table 5.1: Package structure

Module	Responsibility
`crypto/suites.py`	ML-KEM and ML-DSA behind one interface; native (OpenSSL) and pure-Python backends; ECDH/ECDSA baseline
`crypto/kdf.py`	SHAKE-256 hash, HKDF-SHA3-256, session key derivation
`crypto/records.py`	AES-256-GCM records, counters, replay window
`codec.py`	canonical compact binary encoding (signatures need identical bytes on both sides)
`pki.py`	Trusted Authority, certificates, CRL, registry database, identities
`handshake.py`	the four-flight handshake as sans-IO state machines
`certcache.py`	certificate references and the bounded LRU cache
`consensus/agspbft.py`	AGS-PBFT and PBFT replica: normal case, scoring, regrouping, view change
`consensus/sim.py`	discrete-event harness running real replicas with real signatures
`ledger.py`	hash-linked, certified blocks; full-chain verification
`net/secure.py`	secure channels over TCP with length framing
`nodes/`	one process per entity: TA, CAV, RSU, MEC, consensus node, cloud
`sim/`	ns-3 scenario (C++), its Python driver, measurement profiles
`eval/`	benchmarks, attack harness, consensus experiments, ns-3 sweeps, figures, replay viewer

5.2 Cryptographic suites and backends

A *suite* pairs a KEM with a signature scheme. Five are implemented: **DECK** (ML-KEM-1024 + ML-DSA-65, the pairing in our Review 1 design and in [2]), **L1** (512/44), **L3** (768/65), **L5** (1024/87), and **CLASSIC** (ECDH P-256 + ECDSA P-256, as in IEEE 1609.2). DECK pairs a category-5 KEM with a category-3 signature, so its overall strength is category 3, the same as L3.

ECDH is wrapped as a KEM with the DHKEM construction of RFC 9180 [22]: Encaps generates an ephemeral key pair, computes the Diffie–Hellman value with the receiver's public key, hashes it together with both public keys, and sends the ephemeral public key as the "ciphertext". The classical baseline therefore runs through exactly the same handshake code as the post-quantum suites, which makes the comparison like-for-like.

Each algorithm has two interchangeable backends:

- **Native**: the `cryptography` library (version 47 or later), which bundles OpenSSL 3.5 with constant-time ML-KEM and ML-DSA. This is the default and what a deployment would use. The library exposes ML-KEM-768 and ML-KEM-1024 only, so L1's KEM always uses the pure backend.
- **Pure Python**: `kyber-py` and `dilithium-py`, which our Review 1 prototype used. They are readable but slow and, as their authors state, not constant-time.

The two backends are byte-compatible: a test encapsulates with one and decapsulates with the other, and signs with one and verifies with the other.

5.3 Distributed system

Every entity is a separate operating-system process started from one configuration file. A run of the demonstration scenario starts 19 processes: the TA, the cloud, one MEC, eight consensus nodes, three RSUs and five vehicles. Transport is TCP with a 4-byte length prefix per frame; the four handshake flights are frames, and afterwards every frame is one AES-GCM record. Registration and CRL retrieval use plain TCP, which is safe because certificates and CRLs are TA-signed and verified on receipt. Each process writes structured JSON logs, from which an end-to-end report is produced.

The configuration injects faults: a consensus node can return wrong results (`wrong_result`) or stay silent, a node can crash at a given time, the TA can revoke an entity, and vehicles follow a script of events, V2V messages, handovers and reconnections.

5.4 ns-3 simulation

5.4.1 Method

ns-3 3.44 [19] simulates the radio, MAC, backhaul and mobility. The scenario is written in C++ and compiled at run time through the ns-3 Python bindings. ns-3 does not execute the Python cryptography, which would mix wall-clock and simulated time. Instead the real protocol code is run first to produce a *measurement profile*: the exact byte length of every message (the four handshake flights, cached flights, AEAD and signed beacons, events, alerts) and empirical samples of the processing time at each handshake stage, per beacon, and for a consensus commit. The simulation draws from these samples whenever a node processes a message, and each node has a single-server CPU queue.

Table 5.2: ns-3 scenario

Parameter	Value
Radio	IEEE 802.11p, channel 172 (5.86 GHz), 10 MHz, 6 Mb/s, 23 dBm
Propagation	log-distance ($n = 2.2$) + Nakagami fading, $m = 3/1.5/1$ below 50 / 150 / beyond [18]
Detection, CCA	-85 dBm (the 10 MHz value)
Area	1.5 km $\times$ 1.5 km Manhattan grid, 250 m blocks
RSUs	at intersections every 500 m: 9 RSUs, wired to one MEC (1 Gb/s, 2 ms)
Vehicles	50–200; 8–17 m/s; straight 50 %, left or right 25 % each at intersections
Beacons	10 Hz link-layer broadcast (no ACK, no retry)
Fragmentation	1,400-byte chunks; reassembly at the receiver
Handshake	200 ms timeout, 3 retries; NACK for missing Server-Hello chunks
Association	start once $\geq$ 6 of 10 announcements per second are heard; handover to an RSU heard $\geq$ 4 per second better
EDCA	beacons, events, alerts as AC_VO; handshake flights as AC_BE
Backoff	after a failed handshake wait $U(0.5, 1) \times 2^k$ s
Run	30 s simulated, first 5 s excluded; 3 seeds

5.4.2 Simulator pitfalls found and corrected

Five ns-3 defaults distorted results by an order of magnitude before they were corrected, and they belong in any reproduction of this work:

1. **ARP.** The ARP cache queues only three packets per unresolved address, so most fragments of a first ServerHello were silently dropped. 802.11p safety messaging has no ARP, so neighbour caches are pre-populated.
2. **Preamble detection.** ns-3 defaults to the 20 MHz value of −82 dBm; 802.11p's 10 MHz channel uses −85 dBm. The wrong value cut coverage from 99 % to about 80 %.
3. **Unicast beacons** suffered up to 7 MAC retries each; real beacons are broadcasts.
4. **Whole-flight retransmission** after one lost fragment caused a retransmission storm; it was replaced with selective retransmission.
5. **First-contact association.** Starting a 17-frame handshake on the first announcement heard, usually at the edge of coverage, gave 5 % success; association is now gated on link quality.

5.4.3 Certificate caching in the simulation

Mode 1 models the deployment of Section 4.5: RSU certificates are pre-distributed, so every ServerHello carries a reference; after its first session a vehicle sends a reference too, and an RSU that has not seen the vehicle resolves it from the MEC directory at the cost of one backhaul round trip (no airtime). Mode 2, used as an upper bound, starts with all caches warm.

5.4.4 Replay viewer

For demonstration, the simulation can record an event trace (handshake start, session up, failure, link loss, handover, event, alert) and the channel airtime in 0.5 s buckets. A script runs the same scenario (same mobility and seed) for the classical suite, DECK and DECK with caching, and writes a self-contained web page that replays the three runs on the city grid (Figure A.1).

5.5 Evaluation harness

- **Primitives**: 200 timed runs per operation natively and 40 in pure Python, after warm-up runs are discarded.
- **Handshakes**: 100 complete handshakes per suite in memory, recording each stage's processing time and every flight's size, with and without caching.

- **Beacon budget**: per-beacon cryptography plus airtime for AEAD records and for signed broadcasts.
- **Attacks**: 21 attacks run against the real implementation; an attack counts as rejected only if the protocol aborts or drops the message.
- **Consensus**: real replicas with real ML-DSA signatures in a discrete-event loop, for 8–24 nodes, AGS-PBFT with and without shadow validation against PBFT; fault tolerance; view change; score dynamics.
- **ns-3 sweeps**: five suites and DECK with caching at 50–200 vehicles over three seeds, plus single-seed variants.

Timing measurements were taken with the machine otherwise idle, because benchmarks run alongside ns-3 inflated latencies.

5.6 Testing

57 automated tests (`pytest`) cover the codec, FIPS key and signature sizes, key derivation (output lengths and transcript binding), record counters and replay window, backend interoperability, registration (proof of possession, roster, duplicate keys), handshakes for all suites, role checks, certificate caching (savings, fallback, revocation, substituted references, forged HelloRetry), AGS-PBFT (normal case, scoring, regrouping, view change, Byzantine nodes, determinism) and one complete multi-process run of the distributed system.

5.7 Bugs found by our own verification

1. **Consensus stalled about once in 14 runs.** Nodes checked whether the reporting vehicle was registered against the registry copy each had fetched at start-up; a node that started before the vehicle enrolled disagreed with the rest, and with one Byzantine node in a set of four no result reached $2f + 1$. Found by tracing every message of a failing run, fixed by carrying the reporter's TA-signed certificate in the request so validation is a pure function of the request, and verified with 0 failures in 25 runs.
2. **Sequence-number reuse after a view change**: the new leader re-issued prepared requests and then reused their numbers.
3. **PRE-PREPAREs overtaking NEW-VIEW** were dropped as wrong-view; they are now buffered and replayed.

4. **Measurement contention**: timings taken while ns-3 used both cores were inflated and were re-measured.

CHAPTER 6

RESULTS AND DISCUSSION

All numbers in this chapter are generated from the CSV files the evaluation scripts write (`results/`), and all timings were measured on a 2-core cloud virtual machine with the CPU otherwise idle. Absolute timings on other hardware differ; the comparisons between suites do not.

6.1 Cryptographic cost

6.1.1 Primitives

Table 6.1: Primitive operations (ms)

Suite	KEM keygen	encaps	decaps	sign	sign p99	verify
Classical	0.024	0.12	0.091	0.031	0.072	0.12
L1	1.80	2.70	3.63	1.27	3.32	0.16
L1 (pure)	2.29	2.75	3.92	30.2	73.6	8.00
L3	0.14	0.052	0.041	1.30	4.60	0.23
L3 (pure)	3.32	3.80	7.23	50.9	108	11.8
DECK	0.23	0.067	0.052	1.28	4.38	0.23
DECK (pure)	6.87	6.82	7.60	74.0	258	14.7
L5	0.20	0.067	0.052	1.48	5.19	0.42
L5 (pure)	4.52	5.60	7.67	58.9	165	19.1

Mean of 200 runs (native) or 40 runs (pure Python) after warm-up, in ms. L1's KEM is always pure Python because the `cryptography` library exposes no ML-KEM-512.

Table 6.1 gives two findings. First, **signing is the only expensive post-quantum operation, and its variance is intrinsic**: ML-DSA's rejection sampling repeats until a candidate passes, so the 99th percentile is several times the mean in both backends, while verification is deterministic and fast. Second, **the implementation matters more than the algorithm**: the same ML-DSA-65 signs in 1.28 ms natively and 74.0 ms in pure Python, a factor of about 58. Our Review 1 conclusion, that signing might not fit the 100 ms budget, was a property of the pure-Python library, not of ML-DSA.

Table 6.2: Key, signature, certificate and handshake sizes (bytes), with and without certificate caching

Suite	KEM	Signature	pk_{KEM}	sig	cert	bytes	frames	air (ms)	cached	frames	saving
Classical	ECDH-P256	ECDSA-P256	65	70	229	1,043	4	2.05	691	4	33.7 %
L1	ML-KEM-512	ML-DSA-44	800	2,420	3,826	14,373	13	21.3	6,825	8	52.5 %
L3	ML-KEM-768	ML-DSA-65	1,184	3,309	5,355	19,913	17	29.4	9,307	10	53.3 %
DECK	ML-KEM-1024	ML-DSA-65	1,568	3,309	5,355	20,779	17	30.5	10,173	10	51.0 %
L5	ML-KEM-1024	ML-DSA-87	1,568	4,627	7,313	27,329	22	40.1	12,807	12	53.1 %

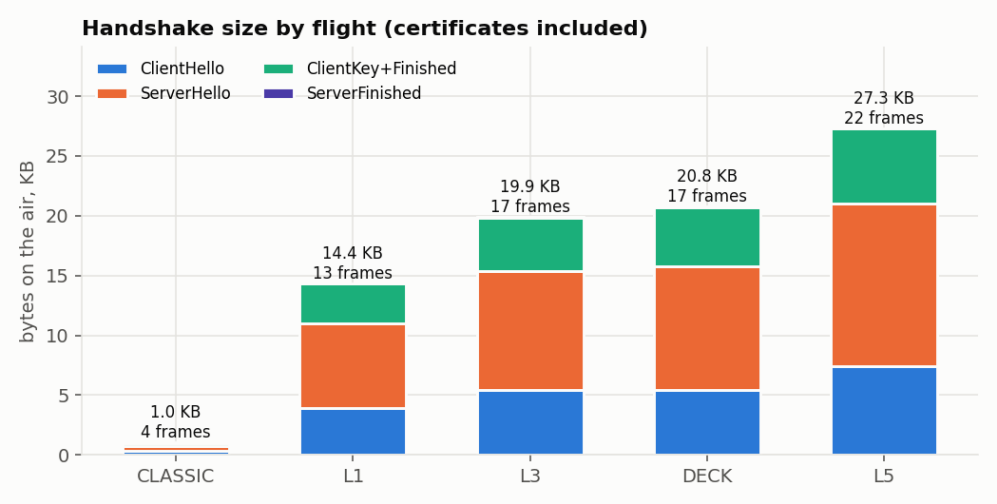


Figure 6.1: Handshake bytes by flight. The ServerHello carries the server's certificate, the ephemeral ML-KEM key and a signature.

6.1.2 Handshake size and compute

A post-quantum handshake is 14–26 times the size of the classical one (Table 6.2, Figure 6.1): 20,779 bytes in 17 frames for DECK against 1,043 bytes in 4 frames, or 30.5 ms of airtime against 2.05 ms. Certificates are the largest single item, 5,355 bytes each. The Review 1 figure of 6,494 bytes left the certificates out.

Computation is small by comparison (Table 6.3, Figure 6.2): about 4.77 ms for a complete DECK handshake natively against 1.16 ms classically, but 209 ms with the pure-Python library. With a native implementation, **the cost of post-quantum V2X is bytes, not CPU time**.

6.1.3 Per-beacon budget

The base paper folds the one-off handshake into its per-message latency. Separated (Table 6.4, Figure 6.3), the per-beacon cost inside a session is the same for every suite: a 135-byte AES-GCM record costing 0.41 ms against a 100 ms interval. Signing every beacon, as IEEE 1609.2 does today, also fits natively (9.51 ms for DECK) but costs 3,472 bytes, 3 frames per beacon, and misses the deadline with the pure-Python library.

Table 6.3: Compute time of one full handshake, both sides (ms)

Suite	native mean	native p95	pure mean	pure p95
Classical	1.16	1.29	–	–
L1	11.6	12.6	106	139
L3	4.77	7.85	178	276
DECK	4.77	7.22	209	319
L5	5.68	8.12	219	303

Compute time of one complete mutual handshake, both sides together, in ms (no network).

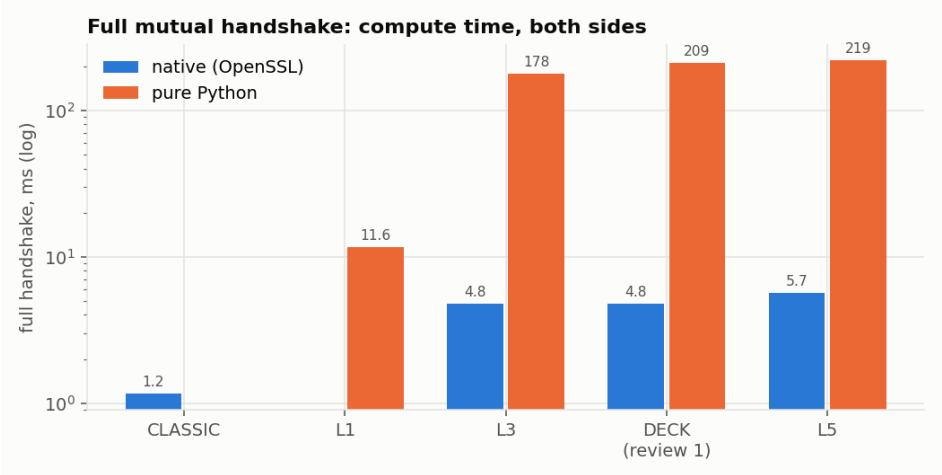


Figure 6.2: Full handshake compute time, native and pure-Python backends (log scale).

6.2 Distributed system

One run of the demonstration scenario (19 processes over TCP on one machine) established 48 secure sessions (10,173–20,779 bytes each, depending on whether certificates travelled as references) and rejected 6 handshakes from the revoked vehicle. Alerts reached vehicles a median 5.4 ms after a critical event was reported, while the event reached the ledger after a median 62 ms: the MEC sends the alert before it submits the event, so **the safety path never waits for consensus**. The Byzantine node (wrong results) was demoted at the first regroup and a candidate promoted; when the leader crashed, a view change installed the next leader and later events committed; after the TA revoked a vehicle, its session was closed and its reconnection rejected. All eight ledger copies were consistent (the crashed leader's copy is a prefix of the others) and verified from genesis.

Table 6.4: Per-beacon cost, p99 crypto plus airtime (ms)

Suite	AEAD in a session	signed broadcast, native	signed, pure Python
Classical	0.40 (135 B)	0.88 (234 B, 1 fr.)	–
L1	0.44 (135 B)	5.55 (2,583 B, 2 fr.)	105 **(misses)**
L3	0.41 (135 B)	9.87 (3,472 B, 3 fr.)	194 **(misses)**
DECK	0.41 (135 B)	9.51 (3,472 B, 3 fr.)	236 **(misses)**
L5	0.47 (135 B)	12.6 (4,790 B, 4 fr.)	184 **(misses)**

p99 of cryptographic processing plus 802.11p airtime per beacon, in ms, against the 100 ms beacon interval.

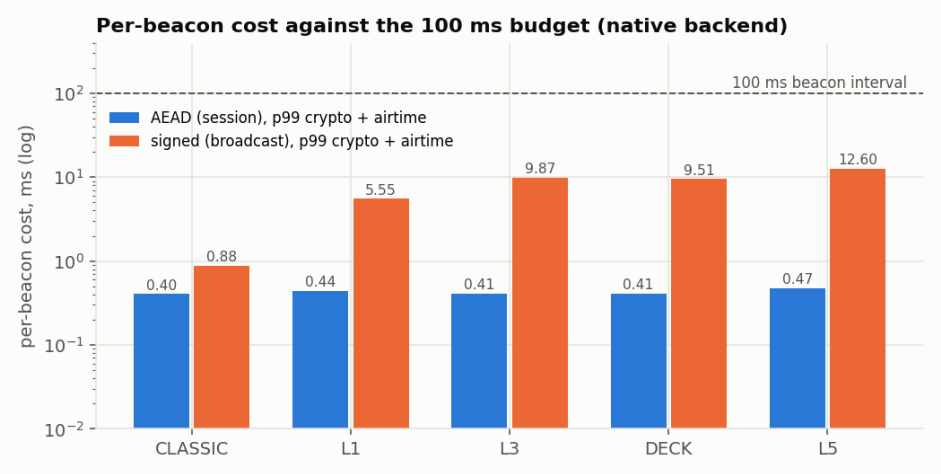


Figure 6.3: Per-beacon cost against the 100 ms beacon interval.

6.3 ns-3: 802.11p with mobility

6.3.1 Handshake success and latency

Tables 6.5 and 6.6 and Figures 6.4–6.5 give the central result. **The classical handshake completes 100 % of the time even at 200 vehicles. Every post-quantum suite degrades with density**, roughly in order of handshake size: L1 (14.4 KB) holds up longest and L5 (27.3 KB) fails first, while L3 and DECK, which differ by less than 1 KB, are within seed-to-seed variation of each other. At 100 vehicles (44 per km^2) DECK completes 74 % of handshakes with a median of 285 ms, against 8 ms classically; at 200 vehicles it completes 27 %. The cause is airtime: handshake fragments push the shared channel into saturation, and losses then trigger retransmissions that add more load.

6.3.2 Other measures

Table 6.7 and Figure 6.6 show the consequence for safety messaging. A vehicle without a session cannot send authenticated beacons, so the fraction of beacon

Table 6.5: Handshakes completed (%) by vehicle density

Vehicles (/km^2)	Classical	L1	L3	DECK	L5	DECK + cache
50 (22)	100	100	99	100	99	100
100 (44)	100	97	70	74	56	98
150 (67)	100	54	40	38	32	80
200 (89)	100	39	27	27	22	57

Handshakes completed, %. Mean of 3 seeds; 1.5 km × 1.5 km Manhattan grid, 9 RSUs, IEEE 802.11p at 6 Mb/s.

Table 6.6: Median handshake completion time (ms)

Vehicles (/km^2)	Classical	L1	L3	DECK	L5	DECK + cache
50 (22)	7	73	88	90	139	54
100 (44)	8	109	288	285	482	73
150 (67)	10	575	651	636	750	197
200 (89)	12	763	760	795	844	417

Median handshake completion time, ms. Mean of 3 seeds; 1.5 km × 1.5 km Manhattan grid, 9 RSUs, IEEE 802.11p at 6 Mb/s.

slots covered by a session falls with the handshake success rate. Up to 100 vehicles alerts stay below 20 ms for every suite, from 5.4 ms (classical) to 16.5 ms (L5) median. Beyond that, a saturated channel delays them too: at 200 vehicles the median is 218.4 ms for DECK and 33.0 ms with caching, against 5.7 ms classically.

6.3.3 Certificate caching

Caching removes 51–53 % of every post-quantum handshake (Table 6.2). In ns-3 (Figure 6.7), with RSU certificates pre-distributed and vehicle certificates resolved from the MEC directory, DECK completes 98 % of handshakes at 100 vehicles instead of 74 %, and 57 % at 200 instead of 27 %; aggregate airtime at 100 vehicles falls from 160 % to 68 %. Caching does not close the gap to the classical suite: a cached DECK handshake is still 10,173 bytes, about ten times a classical one, because the ephemeral ML-KEM key, the ciphertext and two ML-DSA signatures cannot be cached.

6.3.4 Variants and mitigations

Table 6.8 and Figure 6.8 show four further results.

1. **Exponential backoff prevents congestion collapse.** Without it, failed handshakes are retried immediately: DECK completes only 7 % of handshakes (11 % with EDCA alone), and alerts take a median 432.6 ms because they

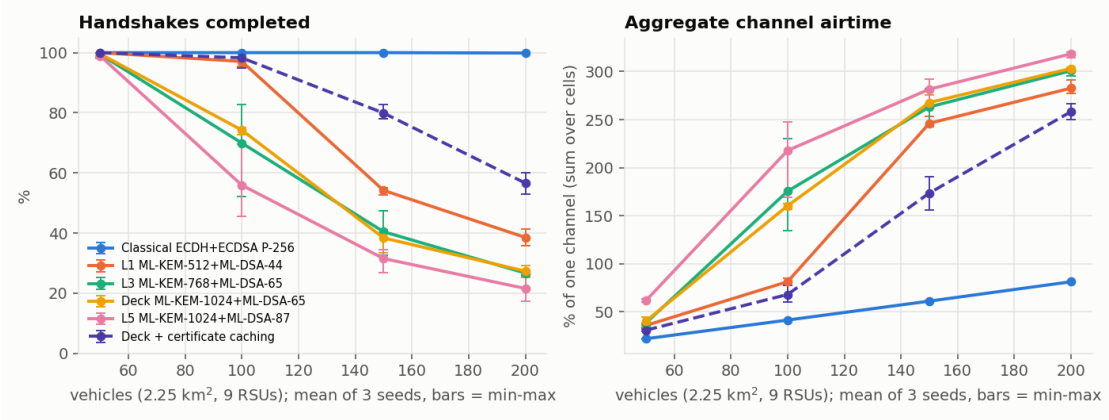


Figure 6.4: Handshakes completed and aggregate channel airtime against density (mean of 3 seeds, bars show min–max).

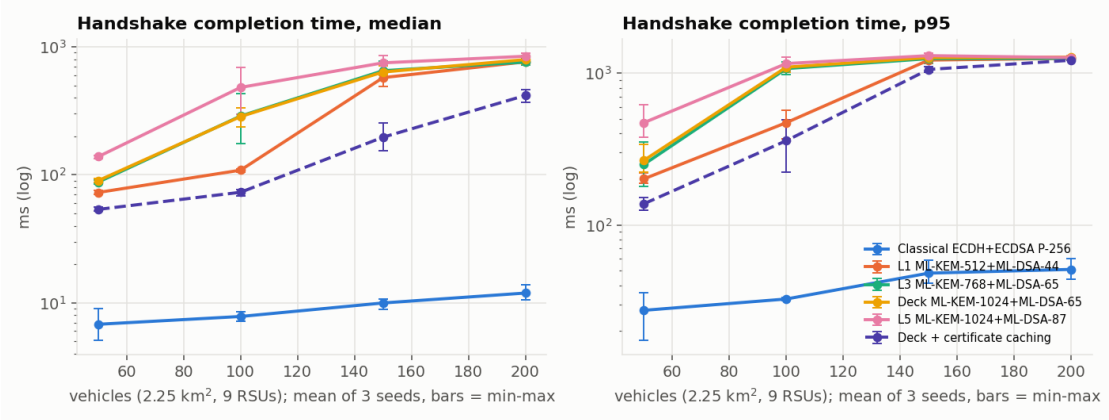


Figure 6.5: Handshake completion time, median and 95th percentile (log scale).

queue behind handshake fragments. With backoff alone DECK completes 77 % and alerts take 8.9 ms.

2. **EDCA priority adds little once backoff is on, at this density.** With both measures DECK completes 75 % of handshakes and the alert median is 7.2 ms, within single-seed noise of backoff alone. Its value is as a safeguard: it keeps safety traffic ahead of handshake fragments whenever the queue builds up.

3. **Signed beacons do not scale with post-quantum signatures.** Signing every beacon (3 frames each) leaves 46 % of beacon slots covered, against 100 % classically. Session-based AEAD beacons are the design that scales.

4. **The implementation still matters in the network.** With processing times from the pure-Python library, DECK completes 69 % of handshakes (native: 75 %) and its median handshake takes 593 ms instead of 283 ms.

Table 6.7: Other measures at 100 vehicles

Measure	Classical	DECK	DECK + cache
Beacon slots with a session (%)	99.9	83.3	97.4
Aggregate airtime (% of one channel)	41	160	68
Handshake time, p95 (ms)	33	1,094	359
Event → alert at the reporter, median (ms)	5.4	7.6	5.6
Ledger commit after the event, median (ms)	11.6	24.6	24.0
Sessions lost (link timeout)	0.0	3.0	1.7

100 vehicles, mean of 3 seeds.

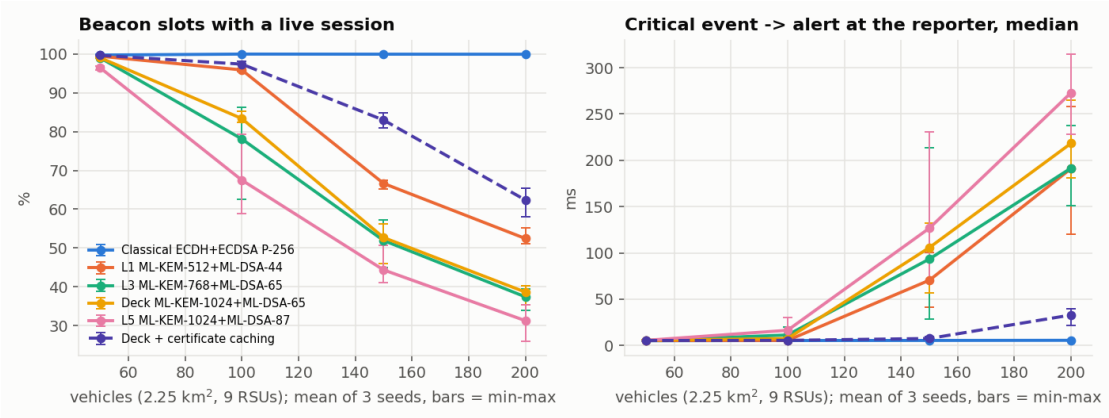


Figure 6.6: Beacon slots with a live session, and the time from a critical event to the alert at the reporting vehicle.

6.4 Consensus

The paper's message saving is real (Table 6.9, Figure 6.9): AGS-PBFT sends 73 % fewer messages than PBFT at 8 nodes and 81–86 % fewer at 12–24 nodes, and at light load it uses 18–25 % less CPU, because only the $3f + 1$ consensus set votes. **Bytes fall much less** (8–30 %): every block carries a commit certificate of $2f + 1$ ML-DSA signatures, and with shadow validation (D7) the candidates' responses as well. Without shadow validation the byte saving is 41–64 %, but candidates can then never earn promotion. **AGS-PBFT is not faster**: replacing PBFT's parallel all-to-all commit with responses to the leader and a certified block adds a sequential hop through one node, so commit latency rises by 41–58 %, and under a burst of 60 requests on 16 nodes AGS-PBFT commits 47 requests per second against 69 for PBFT, and in that run it used 65 % *more* CPU in total (23.3 s against 14.1 s); we have not yet isolated why the light-load saving reverses under a burst. The paper reports only the message count.

Table 6.10 shows PBFT's liveness bound: up to f colluding Byzantine nodes nothing changes, and beyond f the honest nodes can no longer form a $2f + 1$ quorum, so progress stops. No false result was committed in any run, but

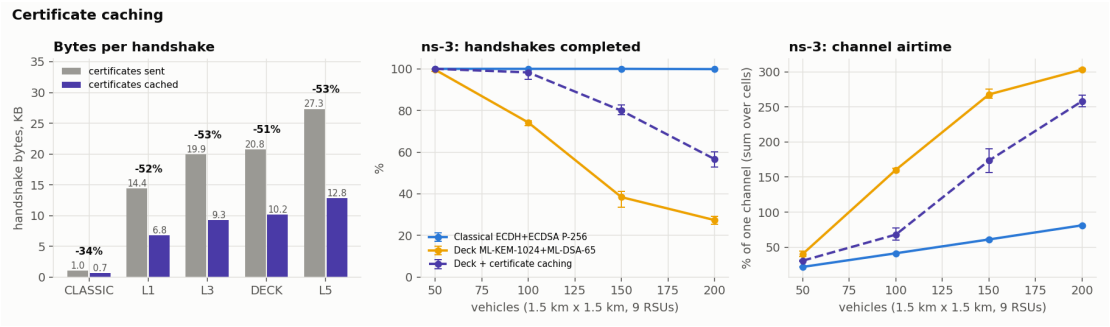


Figure 6.7: Certificate caching: bytes per handshake for every suite, and its effect on DECK in ns-3.

Table 6.8: Single-seed variants at 100 vehicles

Scenario (seed 1)	completed %	p50 ms	coverage %	alert ms	airtime %
DECK, EDCA + backoff (baseline)	75	283	82.3	7.2	157
DECK, neither	7	921	40.1	432.6	339
DECK, EDCA only	11	926	44.3	382.3	330
DECK, backoff only	77	291	84.6	8.9	174
DECK + cache (directory)	100	68	97.9	5.5	66
DECK + cache (all warm)	97	66	98.5	6.0	66
L3 + cache (directory)	100	76	97.1	5.9	71
DECK, signed beacons	23	704	45.9	232.9	342
Classical, signed beacons	100	9	99.9	5.4	54
DECK, pure-Python timings	69	593	73.9	8.6	201

Single-seed variants at 100 vehicles. Coverage: beacon slots with a live session. Alert: event → alert at the reporter, median. Airtime: summed over all cells.

this experiment keeps the leader honest, and an honest leader never certifies a result that differs from its own. With 5 colluders in a set of 7 the colluders could form a quorum of 5 on their own, so a Byzantine *leader* beyond f could commit a false result, exactly as PBFT theory predicts. Recovery after a leader crash (Table 6.11) takes about 2.2–2.4 times the view-change timeout, because the client's retransmission timeout and the backups' view-change timeout run one after the other.

Figure 6.10 shows a weakness of the paper's $\mu \pm \sigma$ rule. The wrong-result node loses 5 points per block and is demoted at the first regroup. The silent node keeps its score of 100 while honest nodes climb, but the wrong-result node's extreme score inflates σ so much that $\mu - \sigma$ stays below 100, and the silent node is never demoted. Without the outlier it is demoted (a regression test checks both cases). A robust statistic, such as the median and the median absolute deviation, would not be distorted by one extreme node.

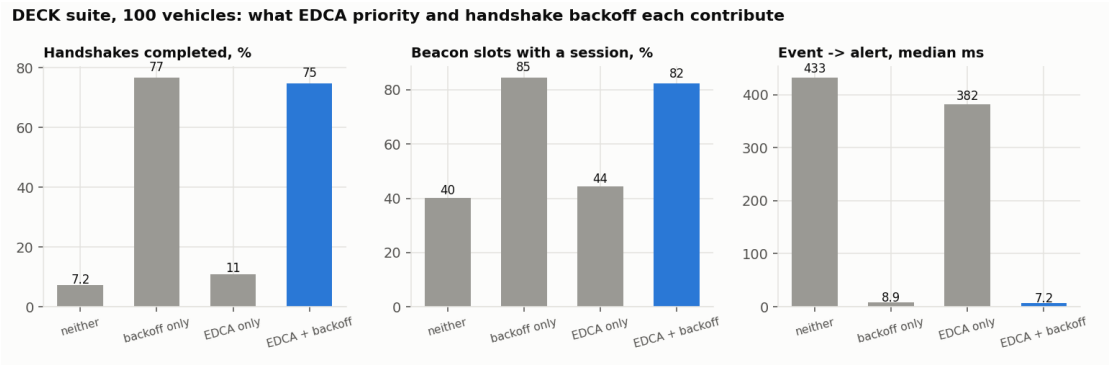


Figure 6.8: Ablation: what EDCA priority and handshake backoff each contribute (DECK, 100 vehicles).

Table 6.9: AGS-PBFT against PBFT

Nodes	$\|CN\|$	msgs/req AGS / PBFT	fewer msgs %	fewer bytes %	no shadow %	latency %	less CPU %
8	4	35 / 128	73	8	41	+49	18
12	4	47 / 288	84	23	64	+46	25
16	7	89 / 512	83	25	57	+41	18
20	10	149 / 800	81	25	53	+43	23
24	10	161 / 1,152	86	30	63	+58	24

Real ML-DSA-65 signatures, 1 ms links, 20 requests at light load. Percentages are AGS-PBFT relative to PBFT; latency is the median commit latency.

6.5 Security

All 21 attacks in Table 6.12 were rejected. Client impersonation with a copied certificate would succeed against the Review 1 design, which had no client signature (F2).

6.6 Answers to the research questions

1. **Can the framework be implemented with the standardised algorithms?** Yes, after the corrections D1–D5 and F1–F7 and the design decisions D6–D9; the result runs as a distributed system and withstands the tested attacks.
2. **Does post-quantum protection fit the 100 ms budget?** Per message, yes, at every security level: 0.41 ms per beacon inside a session, or 9.51 ms for a natively signed beacon. A full handshake costs about 4.77 ms of computation natively.
3. **How does the channel behave as density grows?** The handshake is the problem: at 100 vehicles DECK completes 74 % of handshakes against 100 % classically. Certificate caching and exponential backoff recover much of the loss; a smaller KEM (L3 instead of DECK) costs nothing in security, since DECK's strength is set by its category-3 signature.

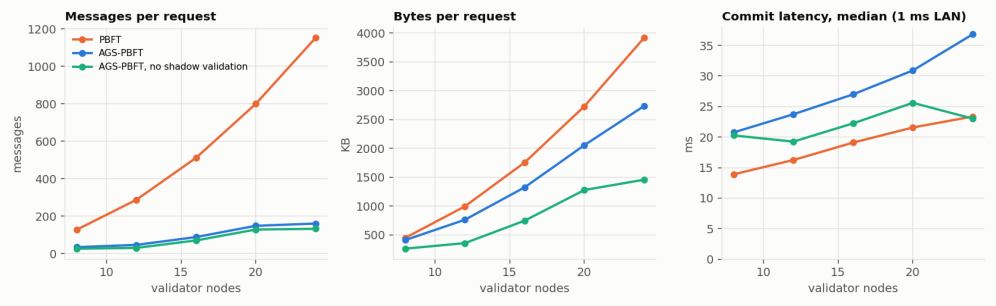


Figure 6.9: Messages, bytes and latency per request against network size.

Table 6.10: Fault tolerance

Byzantine nodes	0	1	2	3	4	5
Requests committed (%)	100	100	100	0	0	0
Safety held	yes	yes	yes	yes	yes	yes

Consensus set 7, f = 2, colluding wrong results, regrouping off.

4. **Does AGS-PBFT beat PBFT?** On messages, yes, and on CPU at light load; on bytes, modestly; on latency, burst throughput and CPU under a burst, no. It never sits on the safety path.

Table 6.11: Recovery after a leader crash

View-change timeout	AGS-PBFT	PBFT
0.5 s	1.21 s	1.19 s
1.0 s	2.22 s	2.17 s
2.0 s	4.72 s	4.68 s

Time from the leader's crash to the first commit in the new view (8 nodes).

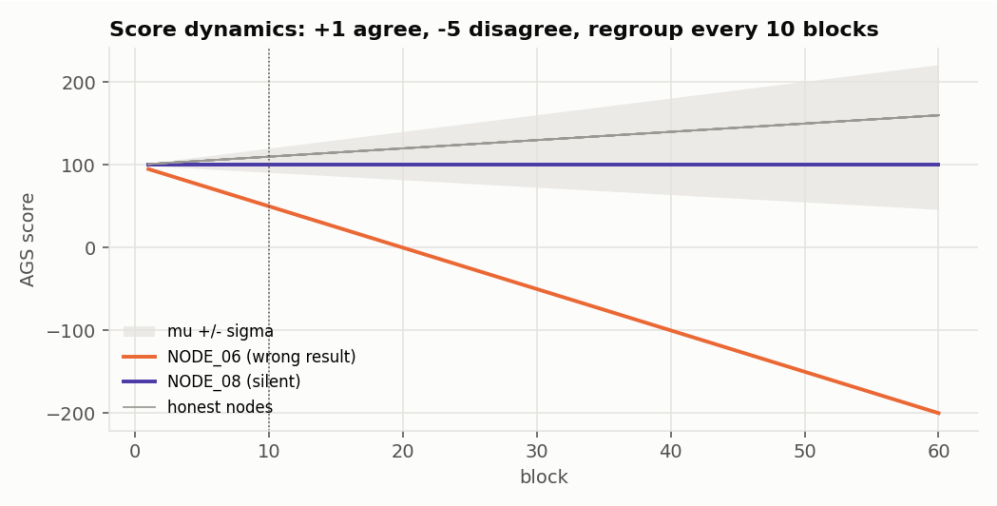


Figure 6.10: Score dynamics with one wrong-result node and one silent node (16 nodes, regroup every 10 blocks).

Table 6.12: Attacks run against the implementation

Attack	Rejected	Mechanism
record tampering (1 bit flipped)	yes	AES-256-GCM tag check fails
record replay	yes	counter already seen in the 64-record window
GCM nonce reuse with Finished	yes	data counters start at 1; 0 is Finished only
ClientHello replay	yes	nonce already in the server's replay cache
stale ClientHello (timestamp -300 s)	yes	outside the 30 s freshness window
client impersonation with stolen certificate	yes	client signature over the transcript fails (deck design had no client signature)
RSU impersonation with stolen certificate	yes	server signature over transcript + pk_kem fails
certificate from a rogue TA	yes	certificate signature does not verify under pk_TA
revoked credential	yes	certificate serial on the signed CRL
cache: claim another vehicle's cached certificate	yes	client signature checked with the referenced certificate's key
cache: revoked certificate still in the cache	yes	cached certificates are re-verified against the current CRL
vehicle posing as an RSU	yes	role is bound into the certificate
MITM substitutes the ephemeral ML-KEM key	yes	server signature covers pk_kem
MITM substitutes the KEM ciphertext	yes	client signature covers c
downgrade to classical suite	yes	suite must match and is inside both signatures
tampered Finished	yes	key confirmation under transcript-bound keys fails
forward secrecy: ephemeral key retained	yes	ephemeral ML-KEM key per session, erased after decapsulation
consensus vote from a non-member	yes	sender has no certificate in the directory
consensus vote spoofing another member	yes	ML-DSA signature does not verify
tampered block (result changed)	yes	leader signature and commit certificate
ledger history rewrite	yes	Pr_B_Hash link broken

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

We built a complete post-quantum security framework for connected vehicles, from registration with a Trusted Authority through a mutually authenticated ML-KEM/ML-DSA handshake and encrypted records to AGS-PBFT consensus over a verifiable ledger, and ran it as a distributed system, in an 802.11p simulation with mobility, and under attack. Implementing the base paper exposed defects that prevent it from working as published, the most serious being a consensus layer that signs with a KEM; implementing our own Review 1 design exposed the lack of client authentication. Both were corrected and the corrections tested.

The measurements answer the project's question. Post-quantum cryptography fits the vehicular latency budget per message, at every NIST security level, once it is implemented natively: a beacon inside a session is a 135-byte AES-GCM record whatever the suite, and a full handshake takes about 4.77 ms of computation. The binding constraint is size. A post-quantum handshake is about twenty times larger than a classical one, and on a shared 6 Mb/s channel that is what fails as density grows: at 100 vehicles in our scenario DECK completes 74 % of handshakes, against 100 % classically. Protocol design recovers much of the loss. Certificate caching halves the handshake and raises completion to 98 %, and exponential backoff after a failed handshake prevents congestion collapse. Safety warnings do not wait for consensus at all, and AGS-PBFT saves messages, and CPU at light load, but not time.

7.2 Limitations

- **One machine.** The distributed system runs on localhost, so its latencies include TCP over loopback, not radio. Radio effects come only from ns-3.
- **ns-3 replays measured costs.** The simulation uses measured message sizes and processing-time distributions rather than executing the cryptography.
- **No WAVE module.** The ns-3 Python package provides the 802.11p PHY and MAC but no WAVE/OCB module, so the ad-hoc MAC is used. Ad-hoc mode also has no association, so behaviour is close.

- **Certificate caching relies on infrastructure we model.** The edge directory is the TA registry in the distributed system and one backhaul round trip in ns-3. A vehicle's first handshake still sends its certificate in full.
- **Linkability.** Certificates and certificate references identify a vehicle across handshakes; pseudonym certificates would be needed for location privacy.
- **V2V through RSUs is not end-to-end.** Relaying RSUs see the plaintext, as in the original design.
- **No formal verification** of the handshake.

7.3 Future work

- **Smaller handshakes.** KEM-based authentication in the style of KEMTLS, which replaces the server's signature with a KEM, or certificates signed with a scheme that has smaller signatures, would attack the remaining size directly.
- **Robust regrouping.** Replace AGS-PBFT's $\mu \pm \sigma$ rule with the median and median absolute deviation, so that one outlier cannot shield a silent node.
- **Formal analysis** of the handshake, including certificate caching, with a protocol verifier such as ProVerif or Tamarin.
- **Pseudonymous certificates** compatible with caching.
- **Realistic mobility** from a traffic simulator on a real road network, and evaluation on vehicular hardware.

BIBLIOGRAPHY

[1] A. M. Aslam, A. Bhardwaj and R. Chaudhary, "Quantum-resilient blockchain-enabled secure communication framework for connected autonomous vehicles using post-quantum cryptography," *Vehicular Communications*, vol. 52, art. 100880, 2025, doi:10.1016/j.vehcom.2025.100880.

[2] A. Olushola and S. P. Meenakshi, "[article title]," *Frontiers in Physics*, vol. 13, art. 1723966, 2025.

[3] National Institute of Standards and Technology, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203, Aug. 2024.

[4] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard*, FIPS 204, Aug. 2024.

[5] National Institute of Standards and Technology, *Transition to Post-Quantum Cryptography Standards*, NIST IR 8547 (initial public draft), Nov. 2024.

[6] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.

[7] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th ACM Symposium on Theory of Computing*, 1996, pp. 212–219.

[8] M. Mosca, "Cybersecurity in an era with quantum computers: will we be ready?" *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.

[9] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler and D. Stehlé, "CRYSTALS-Kyber: a CCA-secure module-lattice-based KEM," in *Proc. IEEE European Symposium on Security and Privacy*, 2018, pp. 353–367.

[10] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler and D. Stehlé, "CRYSTALS-Dilithium: a lattice-based digital signature scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 238–268, 2018.

[11] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *Journal of the ACM*, vol. 56, no. 6, art. 34, 2009.

[12] A. Langlois and D. Stehlé, "Worst-case to average-case reductions for module lattices," *Designs, Codes and Cryptography*, vol. 75, no. 3, pp. 565–599, 2015.

[13] E. Fujisaki and T. Okamoto, "Secure integration of asymmetric and symmetric encryption schemes," in *Advances in Cryptology – CRYPTO '99*, LNCS 1666, 1999, pp. 537–554.

[14] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 1999, pp. 173–186.

[15] E. Androulaki et al., "Hyperledger Fabric: a distributed operating system for permissioned blockchains," in *Proc. 13th EuroSys Conference*, 2018, art. 30.

[16] *IEEE Standard for Wireless Access in Vehicular Environments—Security Services for Application and Management Messages*, IEEE Std 1609.2-2022.

[17] *IEEE Standard for Information Technology—Part 11: Wireless LAN MAC and PHY Specifications, Amendment 6: Wireless Access in Vehicular Environments*, IEEE Std 802.11p-2010.

[18] M. Torrent-Moreno, D. Jiang and H. Hartenstein, "Broadcast reception rates and effects of priority access in 802.11-based vehicular ad-hoc networks," in *Proc. 1st ACM International Workshop on Vehicular Ad Hoc Networks (VANET)*, 2004, pp. 10–18.

[19] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," in *Modeling and Tools for Network Simulation*, Springer, 2010, pp. 15–34.

[20] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, 2018.

[21] S. Santesson and H. Tschofenig, *Transport Layer Security (TLS) Cached Information Extension*, RFC 7924, 2016.

[22] R. Barnes, K. Bhargavan, B. Lipp and C. Wood, *Hybrid Public Key Encryption*, RFC 9180, 2022.

[23] H. Krawczyk and P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, 2010.

[24] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/-Counter Mode (GCM) and GMAC*, NIST SP 800-38D, 2007.

[25] S. Kent, *IP Encapsulating Security Payload (ESP)*, RFC 4303, 2005.

APPENDIX A

RUNNING THE SYSTEM

A.1 Installation

```
python -m venv .venv
.venv\Scripts\activate # Linux/macOS: source .venv/bin/
   activate
pip install -r requirements.txt # cryptography>=47, kyber-py,
   dilithium-py, ...
```

The ns-3 experiments need Linux or WSL2:

```
pip install -r requirements.txt -r requirements-ns3.txt
```

A.2 Commands

Table A.1: Commands

Command	Purpose
`python scripts/demo_protocol.py`	the protocol step by step, with attacks
`python scripts/run_network.py`	the 19-process distributed system
`python -m pytest`	the 57 tests
`python scripts/run_evaluation.py`	benchmarks, attacks, consensus, figures
`python scripts/run_ns3_sweeps.py`	ns-3 density sweep and variants
`python scripts/run_ns3_sweeps.py -seeds 2 3`	extra seeds
`python scripts/make_viewer.py`	the replay viewer (`results/viewer.html`)
`python scripts/make_tables.py`	this report's tables and numbers

A.3 Replay viewer

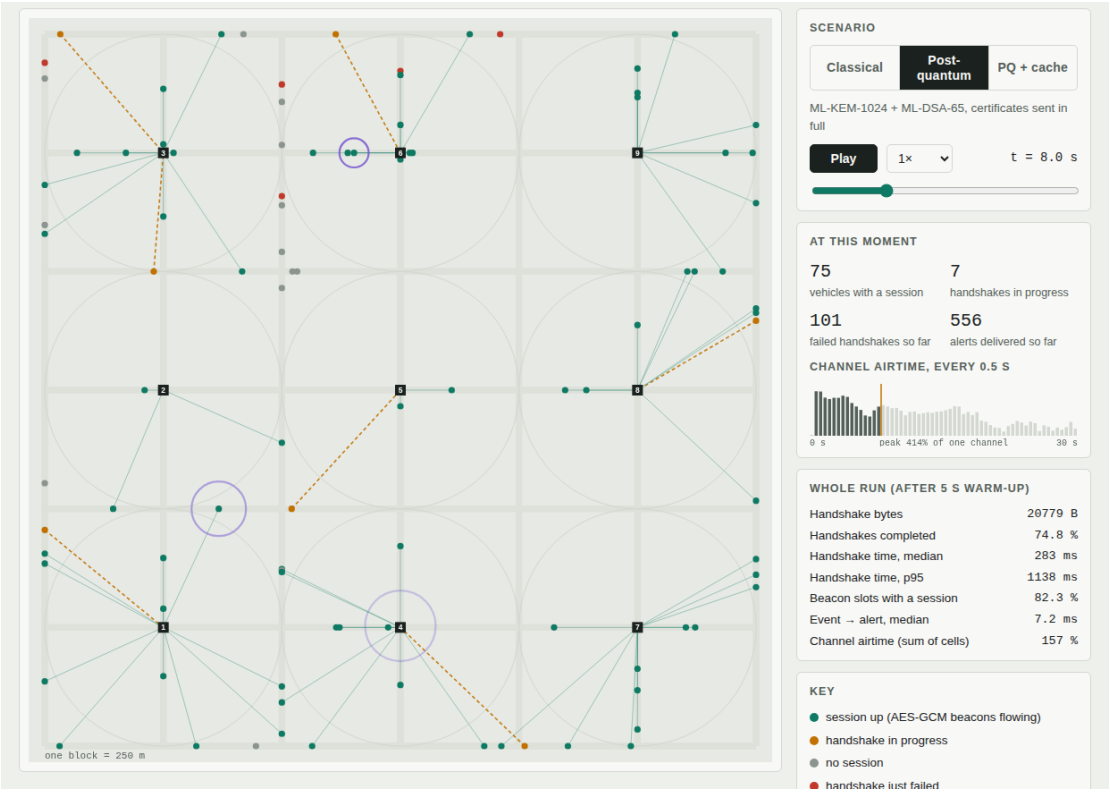


Figure A.1: The replay viewer: vehicles (dots) on the 1.5 km grid, RSUs (squares), handshakes in progress (dashed) and sessions (solid), with the channel airtime over time. The same run can be switched between the classical suite, DECK and DECK with certificate caching.